

Pro task 3 — fix issue #158, the flaky durability test

- **Assigned:** 2026-07-27T21:35-04:00
- **Status:** ASSIGNED (not started)
- **Branch to create:** `worker/pro/158-flaky-durability-test`
- **Base:** fresh `origin/main` (task 2's PR #159 is already merged into it)
- **Worktree:** `/home/tom/projects/Git-Vista-pro`
- **Issue:** #158
- **Effort:** this is a concurrency bug hunt, not mechanical work. Expect to spend most of your time reproducing and diagnosing, and very little writing the fix.

Step 0 — reset your worktree

From inside `/home/tom/projects/Git-Vista-pro`:

```
git fetch origin main
git checkout -b worker/pro/158-flaky-durability-test origin/main
```

Do NOT delete `worker/pro/m1.14-ci-gates` or any other branch. Standing rule.

The problem

`planner::lifecycle_suite::a_finished_operation_is_durable_by_the_time_the_request_returns` (`crates/git-vista-server/src/planner/lifecycle_suite.rs:306-323`) fails intermittently in CI and effectively never locally.

```
thread '...' panicked at crates/git-vista-server/src/planner/lifecycle_suite.rs:
321:5:
assertion `left == right` failed
  left: Failed
 right: Succeeded
```

Observed frequency: 3 CI failures out of 4 observed runs across three different PRs (#156, #157, #159) — including on PRs that changed **only documentation and CI YAML**, so it is definitively not change-induced. Locally it passed 15/15 consecutive runs at `--test-threads=1`, and passed in isolation every time it was tried.

It has now forced two `--admin` merges past a red check. That is the real cost: it is training everyone, including future sessions, to wave through red CI. Read issue #158 in full (gh issue view 158) before starting.

What is already known — do not re-derive this

- **Not git identity.** `seeded_repo` sets `user.email/user.name` repo-locally at `lifecycle_suite.rs:46-47`.

- **The assertion is on line 321, row.1.state.** `left: Failed` means the operation reached a genuine terminal `Failed` state — the journal row was found and its id matched. So this is not a "row missing" or "read too early" bug. **The operation really failed.** That is the single most important clue and it rules out most timing-of-read theories.
- **Two process-global singletons are shared by every test in this binary:**
- `static REGISTRY: OnceLock<StdMutex<Registry>>` — `operations.rs:272`
- `static DB: OnceLock<StdMutex<Connection>>` — `durable.rs:103`, with a single shared `TEST_DB_DIR` at `durable.rs:176`
- **42 `#[tokio::test]` functions live in this one binary** (26 in `contract_suite.rs`, 9 in `lifecycle_suite.rs`, 7 in `coordination_suite.rs`), and cargo runs them concurrently by default. CI's runner has different core count and much slower I/O than the dev box, which is the most likely reason the window opens there and not here.
- The suite's own comment at `lifecycle_suite.rs:84-86` already flags the process-global registry as a known hazard: "The registry is process-global and shared with every other test in this binary, so each test names its own key." Each test namespaces its idempotency **key** — but not the DB, not the mutation guard.
- Relevant machinery to read: `crate::durable::recover` (`durable.rs:364`), the per-repository mutation guard from ADR 0019 (`coordinator.rs`), and `refuse_if_git_busy` (`coordinator.rs:103-113`).

Your job, in order

1. **Reproduce it.** This is most of the work and you should not skip to a fix without it. Locally it hides at low parallelism, so raise the pressure: run the whole binary repeatedly with high `--test-threads`, under `nice/load`, or with the suite looped. Something like `for i in $(seq 1 40); do cargo test -p git-vista-server --bin git-vista-server -- --test-threads=16 || break; done` is a starting point, not a prescription. If you genuinely cannot reproduce locally after real effort, say so and move to step 2 using CI as the reproducer — but say it plainly rather than pretending.
2. **Find out WHY the operation fails**, not just that it does. Right now the assertion tells us the final state and nothing about the cause. Capture the failure reason — the operation's error text, the git stderr, whatever the record carries. **Even if you cannot fix the race, landing a change that makes the next CI failure self-diagnosing is a real and acceptable result.** Say so in your result if that is where you end up.
3. **Fix the root cause.** Likely candidates, in the order I would try them — but follow your evidence, not this list: - cross-test contention on the shared mutation guard (one test's repo lock or busy-check refusing another's operation), - the shared SQLite connection under concurrent writes (lock contention, `SQLITE_BUSY`), - shared temp-dir or path collision between tests.
4. **Prove the fix.** A flaky test is only fixed when it stops being flaky — one green run proves nothing. Run the full binary at high parallelism many times in a row and report the count. Then push and confirm CI is green.

Prefer a root-cause fix over hiding it

Do **not** "fix" this by adding `#[serial]`, a sleep, a retry, or by marking the test `#[ignore]`. Those make the symptom disappear while leaving the actual defect — quite possibly a real concurrency bug in the operation/journal path that also affects production, where two operations on different repositories genuinely do run concurrently. If after real investigation you conclude the correct fix genuinely is test-level isolation, that is an acceptable answer — but argue for it with evidence in `pro-result.md`, do not reach for it first.

Allowed files

- `crates/git-vista-server/src/planner/lifecycle_suite.rs`
- `crates/git-vista-server/src/durable.rs`
- `crates/git-vista-server/src/operations.rs`
- `crates/git-vista-server/src/coordinator.rs`
- Other files in `crates/git-vista-server/` **only if your evidence leads there** — say which and why in your result.

Forbidden

- `crates/git-vista-server/src/git_cmd.rs` — the Max session's #66 chokepoint work starts here. If your evidence points into it, STOP and report in `pro-result.md` rather than editing.

FENCE LIFTED 2026-07-27T22:30 — `crates/git-vista-server/src/planner.rs` is now **ALLOWED**. It was forbidden because #66 was expected to rewrite it imminently; #66's design has since failed its third refutation round, so no implementation is starting there for a while and the fence was protecting nothing. You correctly stopped at it — follow the root cause in now. - `crates/git-vista/**` (frontend), `crates/git-vista-protocol/**`. - `docs/adr/**`, `docs/superpowers/specs/**`, `handoff.md`, `design-docs/**`. - `main`, and any branch but your own. Never force-push, never delete a branch.

Acceptance criteria

- [] The root cause is identified and stated in one clear paragraph — what actually made the operation fail.
- [] The fix addresses that cause, or (acceptable fallback) the failure is made self-diagnosing and you say explicitly that the race remains.
- [] The full test binary passes **at least 25 consecutive runs at high parallelism** locally. Report the exact command and the count.
- [] CI is green on your PR, including the `Core (check + test)` job that has been red on the last three PRs.
- [] No `#[serial]`, no sleep, no retry, no `#[ignore]` — or an evidence-backed argument for why one of them is genuinely correct.
- [] `./dev gate` green.
- [] PR open against `main`, body says `Closes #158`.

Required commands, paste real output

```
./dev gate  
<your repeated-run loop, with the pass count>  
gh pr checks <your PR number>  
git log --oneline main..worker/pro/158-flaky-durability-test
```

Checkpoint yourself, and more often as your budget drains

You are your own checkpointer — the orchestrator's runs against the main checkout only, and must not touch your worktree while you are committing in it.

- Commit and push whenever a meaningful step lands: a reproduction achieved, a hypothesis ruled out, a diagnosis written down. Not only at the end. `wip(#158): <what changed>` is fine. A messy WIP branch is cheap; redone investigation is not.
- **Past ~70% of your budget used, commit after every substantive step.**
- **Past ~85%, write your findings into `pro-result.md` FIRST and commit that,** before attempting anything further. A diagnosis that reaches the orchestrator is worth more than a fix that dies uncommitted. Your Pro bucket is smaller than the orchestrator's — it can end mid-sentence.

Hard rules

1. Work only inside `/home/tom/projects/Git-Vista-pro`. Never `cd` to `/home/tom/projects/Git-Vista` — another session owns that checkout's index.
2. Commits as `claude_2010` with `262510778+tom2025b@users.noreply.github.com`, set `per-commit`.
3. Never delete a branch. Never force-push. Never touch host port 8080, and never start or restart the `git-vista` server.
4. Sign artifacts `thomas2010` with a real ISO timestamp.
5. If your evidence leads into `git_cmd.rs` or `planner.rs`, stop and report — do not edit them.
6. When done, overwrite `/home/tom/projects/Git-Vista/.claude/parallel/pro-result.md` and set `worker.status` to `"done"` in `state.json` in that same directory.

Deliverables

1. Commits on `worker/pro/158-flaky-durability-test`, pushed.
2. PR against `main` saying `Closes #158`.
3. `pro-result.md` with the root-cause paragraph, the repeated-run count, and real CI output.

Signed: thomas2025 · 2026-07-27T21:35:00-04:00